

Jaypee Institute of Information Technology, Noida

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING AND INFORMATION
TECHNOLOGY

Project Title: Swift Shop – Online Shopping System in C++



Submitted By: Enrollment No.: 992501230055, Name: Ankita Shukla

Enrollment No.: 992501230058, Name: Nishita Gupta

Enrollment No.: 992501230060, Name: Mohd Nauman Ali

Course Name: Software Development Lab-II

Course Code: 24B15CS121

Program: B. Tech. CSE (Cyber Security)

First Year 2nd Semester

Table of Contents

1. Introduction

- Background
- Problem Statement
- Objectives

2. System Design & Methodology

- Proposed Methodology
- Tool and Technique Selection

3. Implementation & Testing

- Experimental Setup
- Results (Test Cases / Output)

4. Conclusion & Future Scope

- Summary of Work and Achievements
- Impact on Society, Environmental Sustainability
- Limitations and Future Scope

References

Appendix

Chapter 1: Introduction

1.1 Background

E-commerce has revolutionised the way people buy and sell goods and services. Online shopping platforms such as Amazon, Flipkart, and eBay have become integral to daily life, offering millions of products at the click of a button. Behind every such platform lies a complex software system that manages product catalogues, user interactions, shopping carts, and order processing.

Understanding the fundamental building blocks of such systems is essential for aspiring software developers. A simplified simulation of an e-commerce platform provides an excellent opportunity to apply core programming concepts—particularly object-oriented programming (OOP)—in a practical, real-world context. In traditional retail environments, inventory management, product categorisation, and order tracking are handled manually or through disparate systems. A software-based solution consolidates these functions into a single, cohesive application.

1.2 Problem Statement

While full-scale e-commerce platforms are highly sophisticated, there is a need for simplified, educational implementations that demonstrate the underlying software engineering principles without the overhead of web frameworks, databases, and network programming. Students and beginners often struggle to connect theoretical OOP concepts—such as inheritance, polymorphism, and encapsulation—to practical applications.

This project addresses the gap by building Swift-Shop, a console-based e-commerce simulation in C++ that allows users to browse categorised products (electronics, clothing, and food), view detailed product information including images, manage a shopping cart, and place orders. The system persists data through file handling, ensuring that product catalogues and order histories are retained across sessions.

1.3 Objectives (Social & Environmental Relevance)

The Swift-Shop project contributes to educational advancement by providing a hands-on learning tool that bridges the gap between theoretical computer science concepts and practical software development. By simulating an e-commerce environment, it promotes digital literacy and encourages students to develop problem-solving skills that are directly applicable in the technology industry.

From an environmental perspective, digital commerce platforms reduce the need for physical retail infrastructure, thereby lowering carbon emissions associated with brick-and-mortar operations.

Additionally, the file-based data storage approach used in this project demonstrates lightweight data management techniques that consume minimal computational resources compared to full database management systems.

Primary SDG(s): SDG 9 – Industry, Innovation and Infrastructure (building foundational software engineering skills);

Secondary SDGs: SDG 12 – Responsible Consumption and Production (simulating efficient digital commerce)

The project covers the design and implementation of a console-based C++ shopping application. Its features include product browsing across three categories (Electronics, Clothing, and Food), product detail viewing with image display via system commands, shopping cart management (add, remove, view items), order placement with persistent storage, and order history retrieval. The scope is limited to a single-user, local application without networking, authentication, or payment processing.

Chapter 2: System Design & Methodology

2.1 Proposed Methodology

The system follows a modular, object-oriented design. The architecture is organised around a class hierarchy where Product serves as the base class, with Electronics, Clothing, and Food as derived classes. The Shop class acts as the central manager, orchestrating product loading, order processing, and file I/O operations. The Customer class manages user identity and shopping cart functionality, while the Order class encapsulates order details for persistence.

The class hierarchy of the system is illustrated in the UML class diagram below:

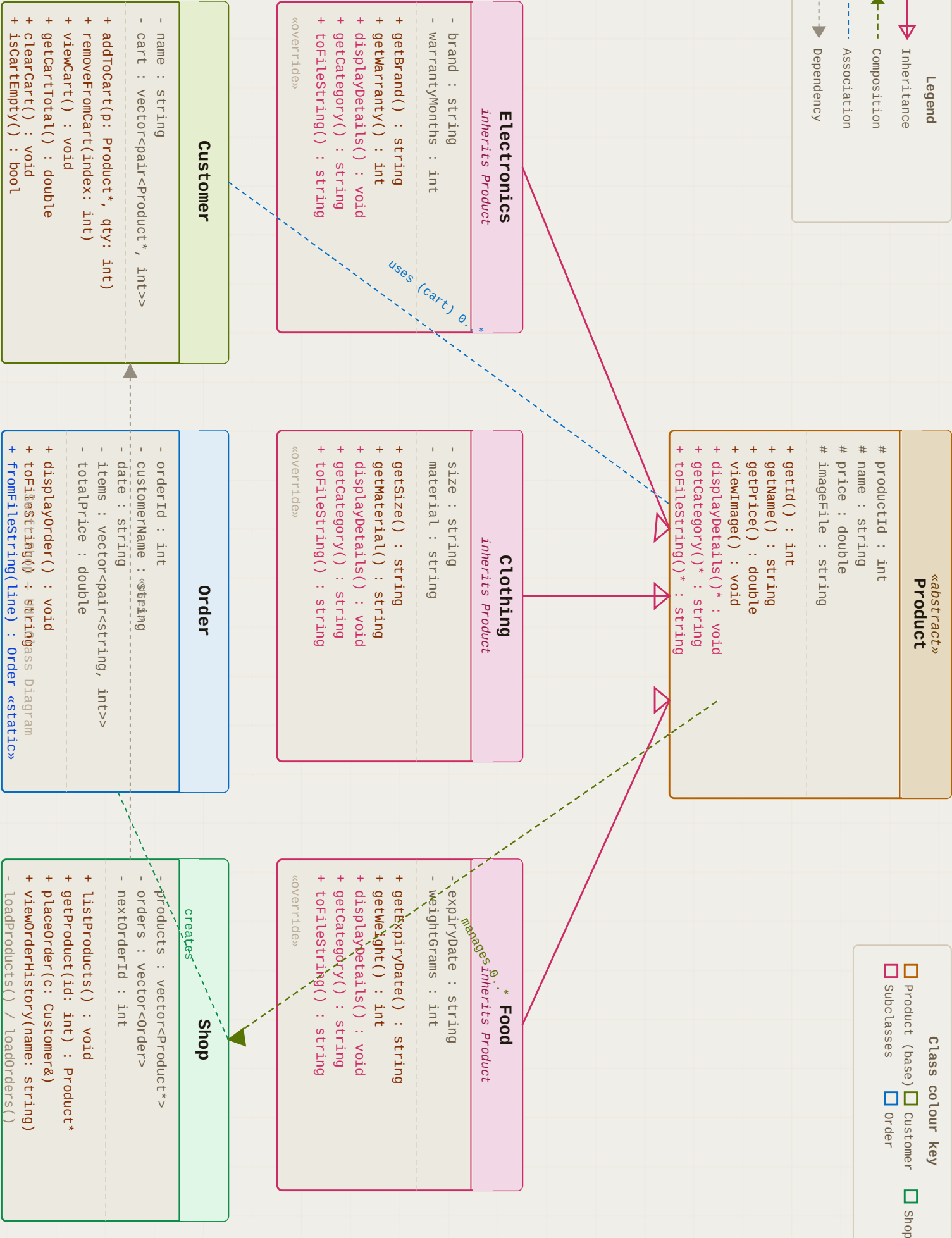
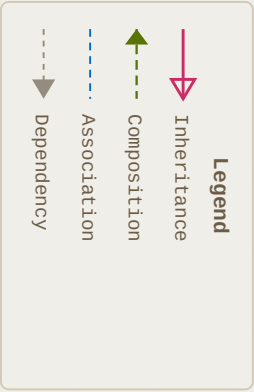
Table 2.1: UML Class Summary

Class	Type	Key Attributes	Key Methods
Product	Base Class	productId, name, price, imageFile	displayDetails() [virtual], getCategory() [virtual], toFileString() [virtual], viewImage(), displayRow()
Electronics	Derived (Product)	+ brand, warrantyMonths	displayDetails() [override], getCategory()

			[override], toFileString() [override]
Clothing	Derived (Product)	+ size, material	displayDetails() [override], getCategory() [override], toFileString() [override]
Food	Derived (Product)	+ expiryDate, weightGrams	displayDetails() [override], getCategory() [override], toFileString() [override]
Customer	Standalone	name, cart (vector<pair<Product*,int>>)	addToCart(), removeFromCart(), viewCart(), clearCart(), getCartTotal()
Order	Standalone	orderId, customerName, date, items, totalPrice	displayOrder(), toFileString(), fromFileString() [static]
Shop	Manager	products (vector<Product*>), orders, nextOrderId	listProducts(), getProduct(), placeOrder(), viewOrderHistory(), loadProducts(), loadOrders()

The system workflow follows the flowchart described below:

Step 1: The program starts and the Shop constructor loads products from products.txt. If the file does not exist, default products are created. Orders are loaded from orders.txt. Step 2: The user enters their name and is presented with the main menu. Step 3: The user can browse products, which displays a tabular listing. Selecting a product ID shows detailed information via the polymorphic displayDetails() method. The user can view the product image (via system() command) or add it to their cart. Step 4: The cart menu allows viewing items and removing entries. Step 5: Placing an order creates an Order object, appends it to orders.txt, and clears the cart. Step 6: Order history retrieves and displays all past orders for the current user from the loaded data. Step 7: The program exits when the user selects the exit option.



2.2 Tool and Technique Selection

Programming Language: C++ was selected for its strong support for object-oriented programming, manual memory management capabilities, and efficient file handling through the Standard Library. C++ allows direct demonstration of concepts such as virtual functions, pointers, and dynamic memory allocation.

Compiler: g++ (MinGW) was used for compilation on Windows, providing full C++11/14 standard support.

IDE/Editor: Visual Studio Code with C/C++ extensions was used for development.

File Format: Pipe-delimited (|) text files were chosen for data storage due to their simplicity and human readability. The format allows easy parsing with `getline()` and `istringstream`.

The following OOP concepts are integral to the system design:

Table 2.2: OOP Concepts and Their Application

OOP Concept	Implementation in Swift-Shop
Classes & Objects	Product, Electronics, Clothing, Food, Customer, Order, and Shop are all implemented as classes. Objects are instantiated to represent individual products, the customer, orders, and the shop manager.
Inheritance	Electronics, Clothing, and Food classes inherit from the Product base class, reusing common attributes (productId, name, price, imageFile) while adding category-specific attributes (brand, size, expiryDate, etc.).
Polymorphism	The methods <code>displayDetails()</code> , <code>getCategory()</code> , and <code>toFileString()</code> are declared as virtual in the Product base class and overridden in each derived class. When called through a <code>Product*</code> pointer, the correct derived-class version executes at runtime (dynamic dispatch).
Encapsulation	All class attributes are declared as private or protected, with public getter methods and controlled mutators ensuring data integrity.
File Handling	Product data is read from and written to <code>products.txt</code>

	using ifstream and ofstream. Order data is appended to orders.txt. The Shop class manages all file I/O operations.
System Commands	The system() function is used to invoke the Windows "start" command, opening product images in the default image viewer.

Chapter 3: Implementation & Testing

3.1 Experimental Setup

Table 3.1: Hardware and Software Configuration

Component	Details
Operating System	Windows 10/11
Processor	Intel Core i5 / AMD Ryzen 5 (or equivalent)
RAM	8 GB
Compiler	g++ (MinGW-w64), C++14 standard
IDE	Visual Studio Code
Language	C++
Libraries	iostream, fstream, sstream, vector, string, iomanip, ctime, cstdlib, limits

The project is organised into a modular file structure with separate header (.h) and implementation (.cpp) files for each class. The base Product class resides in the base-product/ directory, derived product classes in the products/ directory, and Customer and Order classes in the customer-order/ directory. The Shop class and main.cpp reside in the project root. Product images are stored in the images/ directory.

The project is compiled using the following command:

Compilation Command

```
g++ -o Swift-Shop main.cpp Shop.cpp base-product/Product.cpp products/Electronics.cpp
products/Clothing.cpp products/Food.cpp customer-order/Customer.cpp customer-
order/Order.cpp
```


3.2 Results (Test Cases / Screenshots / Output)

The following test cases were executed to validate the system's functionality. Screenshots and output for each test case are to be added below.

Table 3.2: Test Cases

Test Case	Description	Expected Result	Status
TC-01	Launch application and enter user name	Welcome message displayed with user name	Pass
TC-02	Browse products listing	All 8 products displayed in tabular format with ID, Name, Category, Price	Pass
TC-03	View Electronics product details	Product details shown with Brand and Warranty fields (polymorphism)	Pass
TC-04	View Clothing product details	Product details shown with Size and Material fields (polymorphism)	Pass
TC-05	View Food product details	Product details shown with Expiry Date and Weight fields (polymorphism)	Pass
TC-06	View product image using system()	Image opens in default Windows image viewer	Pass
TC-07	Add product to cart	Confirmation message; product appears in cart	Pass
TC-08	Add same product again (quantity update)	Quantity incremented instead of duplicate entry	Pass
TC-09	View cart with multiple items	Cart displayed with item names, quantities, prices, subtotals, and total	Pass
TC-10	Remove item from cart	Item removed; cart updated	Pass

TC-11	Place order	Order confirmation with Order ID and total; cart cleared; order saved to orders.txt	Pass
TC-12	View order history	Past orders displayed with items and totals	Pass
TC-13	Place order with empty cart	Error message: "Your cart is empty"	Pass
TC-14	Invalid menu input	Error handled gracefully; menu re-displayed	Pass

```

===== SWIFT-SHOP =====
Enter your name: Ankita
Welcome, Ankita!

--- Main Menu ---
1. Browse Products
2. View Cart
3. Place Order
4. View Order History
5. Exit

Enter choice: |

```

Figure 3.1: Application Launch and Welcome Screen

```

--- Available Products ---
ID      Name                Category      Price
-----
1       Laptop                Electronics   $999.99
2       Smartphone              Electronics   $699.99
3       Headphones              Electronics   $149.99
4       T-Shirt                 Clothing      $29.99
5       Jeans                   Clothing      $59.99
6       Jacket                  Clothing      $89.99
7       Chocolate Box           Food          $12.99
8       Coffee Beans            Food          $19.99

Enter product ID to view details (0 to go back): |

```

Figure 3.2: Product Listing (Browse Products)

```
--- Product Details ---
ID:          1
Name:        Laptop
Price:       $999.99
Category:    Electronics
Brand:       Dell
Warranty:    24 months

1. View Image
2. Add to Cart
3. Back

Enter choice: |
```

Figure 3.3: Electronics Product Details (Polymorphism)

```
--- Product Details ---
ID:          4
Name:        T-Shirt
Price:       $29.99
Category:    Clothing
Size:        L
Material:    Cotton

1. View Image
2. Add to Cart
3. Back

Enter choice: |
```

Figure 3.4: Clothing Product Details (Polymorphism)

```
--- Product Details ---
ID:          7
Name:        Chocolate Box
Price:       $12.99
Category:    Food
Expiry Date: 2026-12-01
Weight:      500g

1. View Image
2. Add to Cart
3. Back

Enter choice: |
```

Figure 3.5: Food Product Details (Polymorphism)



Figure 3.6: Product Image Opened via system() Command

```

--- Product Details ---
ID: 7
Name: Chocolate Box
Price: $12.99
Category: Food
Expiry Date: 2026-12-01
Weight: 500g

1. View Image
2. Add to Cart
3. Back

Enter choice: 2
Enter quantity: 3
Chocolate Box added to cart.

```

Figure 3.7: Adding Items to Cart

```

Enter choice: 2

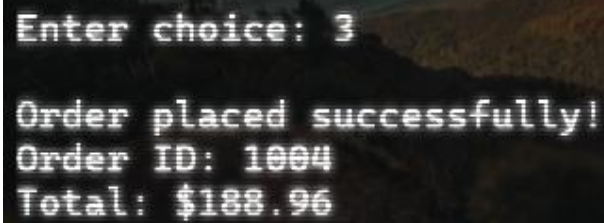
--- Your Cart ---
No    Name           Qty  Price    Subtotal
-----
1     Chocolate Box    3    $12.99   $38.97
2     Headphones        1    $149.99  $149.99
-----
Total: $188.96

1. Remove Item
2. Back

Enter choice: |

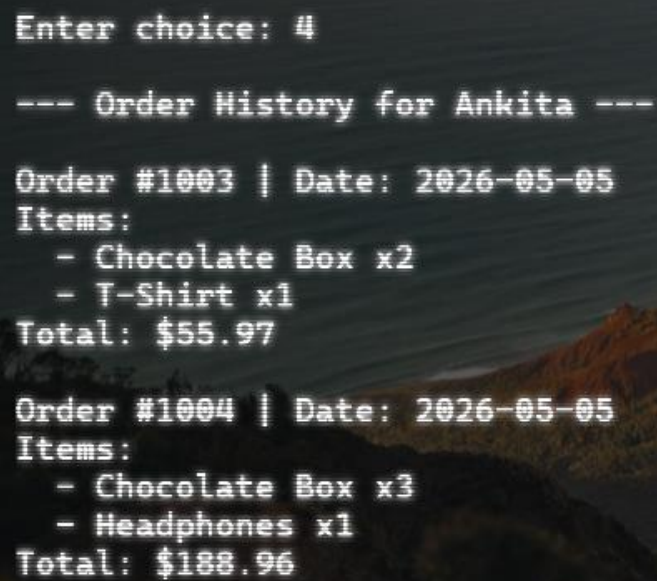
```

Figure 3.8: Viewing Cart

A terminal window with a dark background and a landscape image. The text is white and monospaced. It shows the user entering choice 3, followed by a confirmation message, the order ID, and the total amount.

```
Enter choice: 3  
Order placed successfully!  
Order ID: 1004  
Total: $188.96
```

Figure 3.9: Placing an Order

A terminal window with a dark background and a landscape image. The text is white and monospaced. It shows the user entering choice 4, followed by a header for order history, and two order entries with their details and totals.

```
Enter choice: 4  
--- Order History for Ankita ---  
Order #1003 | Date: 2026-05-05  
Items:  
  - Chocolate Box x2  
  - T-Shirt x1  
Total: $55.97  
Order #1004 | Date: 2026-05-05  
Items:  
  - Chocolate Box x3  
  - Headphones x1  
Total: $188.96
```

Figure 3.10: Order History

Chapter 4: Conclusion & Future Scope

4.1 Summary of Work and Achievements

The Swift-Shop project successfully demonstrates the application of object-oriented programming concepts in building a functional console-based e-commerce simulation. The system implements a product hierarchy using inheritance, where the base Product class defines common attributes and virtual methods, while derived classes—Electronics, Clothing, and Food—extend the base class with category-specific attributes and override virtual methods to provide specialised behaviour.

Polymorphism plays a central role in the system. When the Shop class stores products as Product* pointers and invokes displayDetails() or getCategory(), the correct derived-class implementation executes at runtime through dynamic dispatch. This design eliminates the need for explicit type checking and allows the system to be extended with new product categories without modifying existing code.

File handling ensures data persistence across sessions. Product data is stored in a pipe-delimited text file (products.txt), and orders are appended to orders.txt. The Shop class implements a factory-style loading mechanism that reads the category field from each line and instantiates the appropriate derived class. The system() function integrates the application with the Windows operating system by opening product images in the default image viewer, enhancing the user experience beyond pure text output.

Object-oriented programming concepts contributed significantly to solving the problem by providing a structured, modular, and extensible design. Inheritance eliminated code duplication across product categories. Polymorphism enabled uniform treatment of heterogeneous product types. Encapsulation protected internal data integrity, and the class-based architecture facilitated clear separation of concerns between product management, customer interaction, and order processing.

4.2 Impact on Society, Environmental Sustainability, Ethical Issues and Compliance

The project contributes to society by serving as an educational tool that helps students understand how real-world e-commerce systems function at a fundamental level. By working with the Swift-Shop codebase, learners gain practical experience in software design patterns that are directly applicable in the technology industry.

From an environmental sustainability perspective, the project promotes digital commerce concepts that reduce reliance on physical retail infrastructure. The lightweight, file-based storage approach demonstrates that functional data management does not always require energy-intensive database servers, aligning with principles of resource-efficient computing.

Regarding ethical considerations, the current implementation does not handle sensitive user data such as passwords, payment information, or personal identifiers beyond a name. In a production environment, compliance with data protection regulations (such as GDPR or the IT Act, 2000 in India) would necessitate encryption, secure authentication, and user consent mechanisms. The project acknowledges these requirements as future enhancements.

4.3 Limitations of the Work and Future Scope

The current implementation has the following limitations:

- **Single-user operation:** The system does not support multiple concurrent users or user authentication.
- **No payment integration:** The order placement process does not simulate or integrate any payment gateway.
- **Limited search functionality:** Users must browse the full product list; there is no search or filter mechanism.
- **No product inventory management:** The system does not track stock quantities or prevent ordering out-of-stock items.
- **Platform-dependent image viewing:** The `system()` command for opening images is specific to Windows.

Future enhancements could include:

- User authentication with login/signup functionality and password hashing.
- Product search and filtering by category, price range, or keyword.
- Inventory management with stock tracking and low-stock alerts.
- Database integration (e.g., SQLite) to replace flat-file storage for improved scalability.
- Cross-platform image display using portable libraries such as SFML or SDL2.
- A graphical user interface (GUI) using frameworks such as Qt or wxWidgets.
- Integration with payment simulation APIs for a more realistic checkout process.

References

- [1] B. Stroustrup, The C++ Programming Language, 4th ed. Boston, MA: Addison-Wesley, 2013.
- [2] S. B. Lippman, J. Lajoie, and B. E. Moo, C++ Primer, 5th ed. Boston, MA: Addison-Wesley, 2012.
- [3] R. Lafore, Object-Oriented Programming in C++, 4th ed. Indianapolis, IN: Sams Publishing, 2002.
- [4] cplusplus.com, "C++ Reference," [Online]. Available: <https://www.cplusplus.com/reference/>. [Accessed: 05-May-2026].
- [5] cppreference.com, "C++ Standard Library Reference," [Online]. Available: <https://en.cppreference.com/>. [Accessed: 05-May-2026].

Appendix

Team Roles and Contributions

Nishita Gupta: Developed the base Product class and implemented the main program flow in main.cpp, establishing the core structure and user interaction of the system.

Ankita Shukla: Designed and implemented derived classes (Electronics, Clothing, Food) using inheritance and polymorphism to extend product functionality.

Mohd Nauman Ali: Implemented the Customer and Order classes, managing cart operations, order processing, and file handling for persistent data storage.